OPERATION NIGHTFALL — CTF Project Report

1. Introduction

Operation Nightfall is an Android-based Capture The Flag (CTF) challenge designed to teach reverse engineering, mobile security, and dynamic analysis techniques. The application simulates a spy-themed mission where participants uncover hidden flags using various analysis methods.

2. Objective

The objective of this project is to analyze an Android application and extract nine hidden flags. Each flag represents a specific vulnerability or reverse engineering concept.

3. Methodology

The project involves both static and dynamic analysis techniques. Static analysis includes decompiling APK files and inspecting resources, while dynamic analysis involves runtime instrumentation using tools like Frida.

4. Tools Used

APKTool, JADX, Frida, ADB, Python

5. Flag Analysis

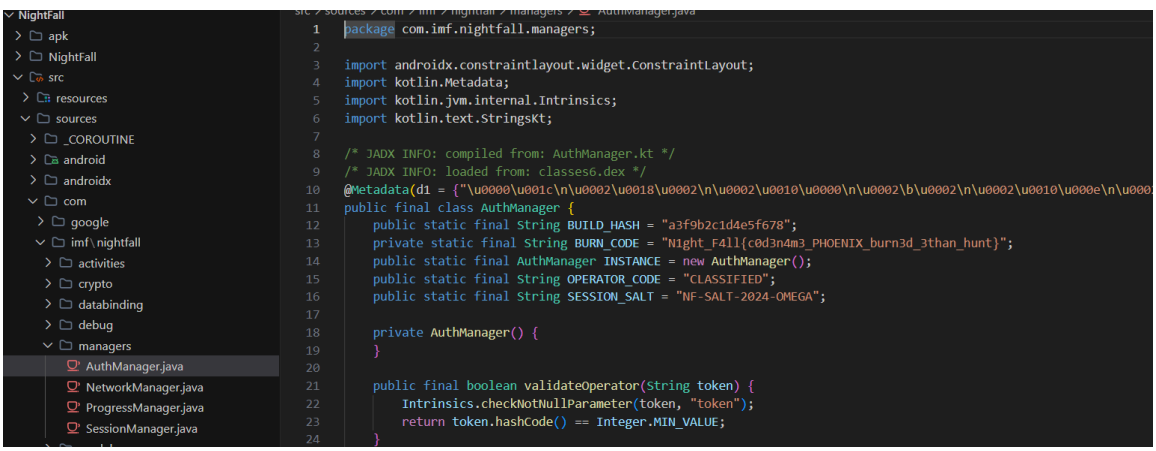
Flag 1 — Dead Drop

Flag: N1ght_F4ll{3th4n_1s_4l1v3_bur13d_1n_str1ngs}
Category: Static Analysis / Base64 | Difficulty: ★
Finding Method: Extract strings.xml using APKTool or JADX.
Concept: Base64 encoding.
Explanation: The flag is stored as a Base64-encoded string in res/values/strings.xml under the key transmission_code. Decompile the APK with apktool, find the encoded string, and decode it with base64 -d to reveal the flag.
Steps: 1) Run: apktool d nightfall.apk 2) Open nightfall/res/values/strings.xml and find transmission_code value 3) Run: echo "TjFnaHRfRjRsbHszdGg0bl8xc180bDF2M19idXIxM2RfMW5fc3RyMW5nc30=" | base64 -d

```
PS C:\Users\abhio\Downloads\NightFall> findstr /i "transmission_code" Nightfall\src\resources\res\values\strings.xml
    <string name="transmission_code">TjFnaHRfRjRsbHszdGg0bl8xc180bDF2M19idXIxM2RfMW5fc3RyMW5nc30=</string>
PS C:\Users\abhio\Downloads\NightFall> powershell -Command "[System.Text.Encoding]::UTF8.GetString([System.Convert]::
omBase64String('TjFnaHRfRjRsbHszdGg0bl8xc180bDF2M19idXIxM2RfMW5fc3RyMW5nc30='))"
N1ght_F4ll{3th4n_1s_4l1v3_bur13d_1n_str1ngs}
PS C:\Users\abhio\Downloads\NightFall>
```

Flag 2 — Burn Notice

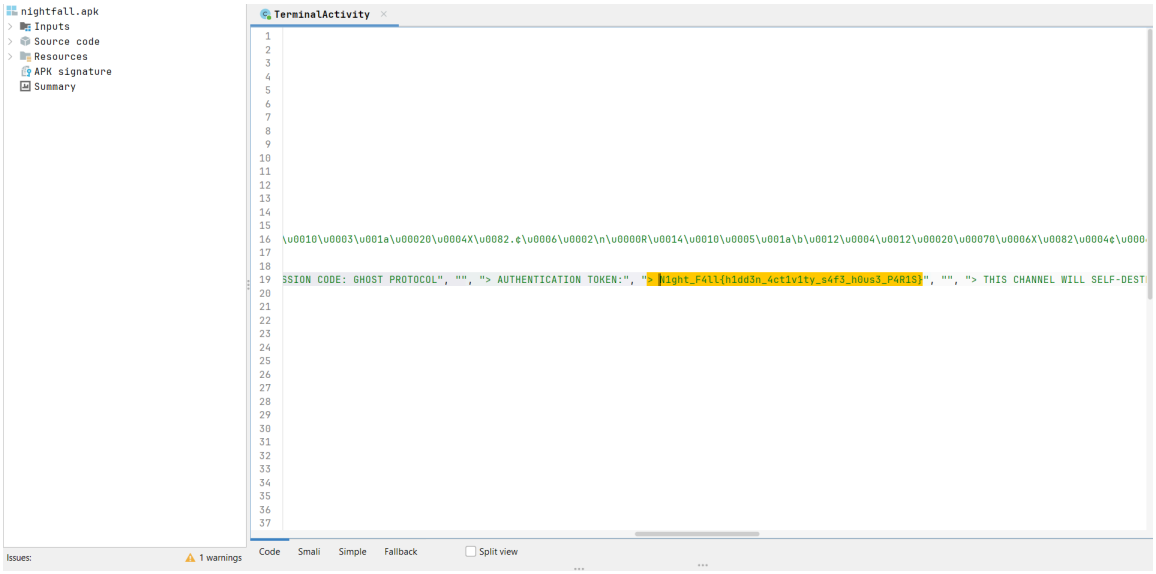
Flag: N1ght_F4ll{c0d3n4m3_PHOENIX_burn3d_3than_hunt}
Category: Hardcoded Constant | Difficulty: ★★
Finding Method: Decompile source code and search for hardcoded strings.
Concept: Hardcoded secrets.
Explanation: The flag is directly embedded in AuthManager.kt as a constant named BURN_CODE. Decompiling the APK with JADX and searching for this constant reveals the flag.
Steps: 1) Run: jadx -d src nightfall.apk 2) Search: grep -r "BURN_CODE" src/ 3) Open AuthManager.kt and read the constant value



Flag 3 — Ghost Protocol

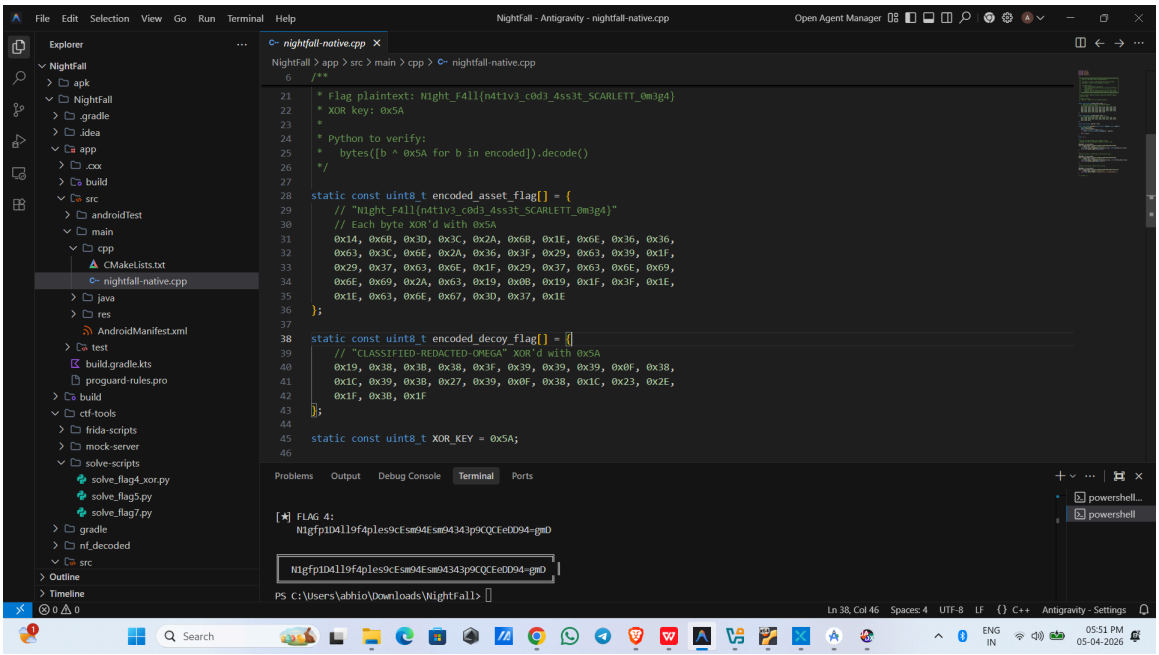
Flag: N1ght_F4ll{h1dd3n_4ct1v1ty_s4f3_h0us3_P4R1S}
Category: Hidden Activity / ADB | Difficulty: ★★
Finding Method: Analyze AndroidManifest and launch hidden activity using ADB.
Concept: Hidden components.
Explanation: TerminalActivity is declared in AndroidManifest.xml but has no launcher intent, making it invisible to the user. The flag is revealed by directly launching it via ADB shell.
Steps: 1) Run: apktool d nightfall.apk and inspect AndroidManifest.xml for unlisted activities

2) Run: adb shell am start -n com.imf.nightfall/.activities.TerminalActivity 3) Read the flag displayed on the terminal screen



Flag 4 — The Asset

Flag: N1ght_F4ll{n4t1v3_c0d3_4ss3t_SCARLETT_0m3g4}
Category: NDK / XOR Native | Difficulty: ★★★
Finding Method: Analyze native .so library using Ghidra or strings.
Concept: Native code reverse engineering.
Explanation: The flag is XOR-encoded inside nightfall-native.cpp and compiled into libnightfall.so. It can be found by running strings on the .so file or using solve_flag4_xor.py to decode the XOR bytes.
Steps: 1) Extract APK and navigate to lib/arm64-v8a/ 2) Run: strings libnightfall.so | grep "N1ght" OR 3) Run: python3 ctf-tools/solve-scripts/solve_flag4_xor.py 4) Alternatively use Ghidra/radare2 to reverse the native XOR routine



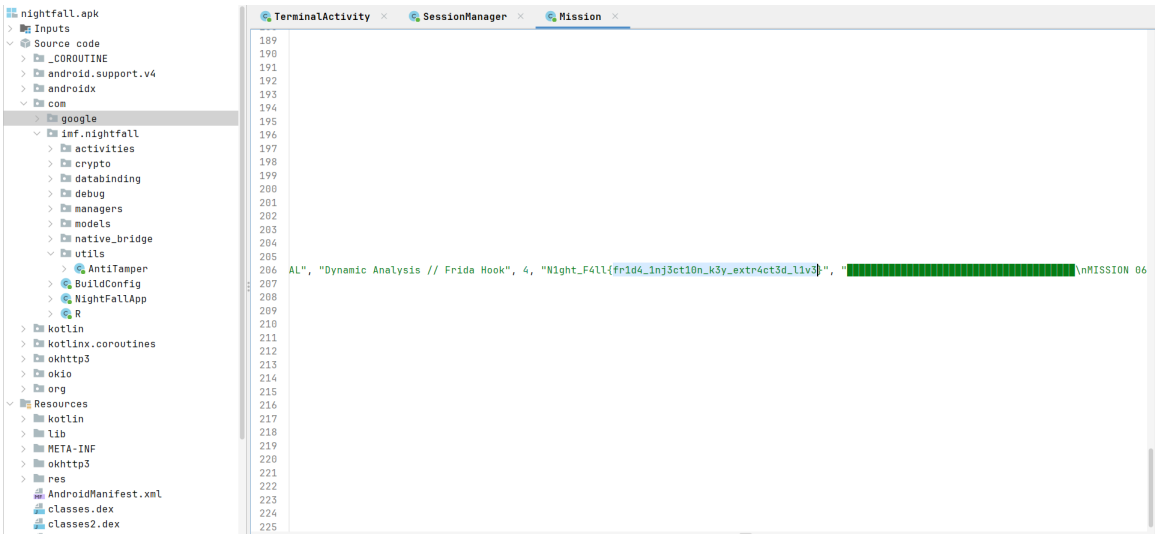
Flag 5 — Cipher Nine

Flag: N1ght_F4ll{r0t13_plus_b64_c1ph3r_n1n3_bl3print}
Category: ROT13 + Base64 | Difficulty: ★★★
Finding Method: Extract raw resource file and decode.
Concept: Layered encoding (ROT13 then Base64).
Explanation: The file res/raw/classified_doc.txt contains a ROT13+Base64 encoded payload. It must be decoded in two steps: first Base64 decode, then apply ROT13 (or vice versa based on order). The solve script automates both steps.
Steps: 1) Decompile APK and extract res/raw/classified_doc.txt 2) Run: python3 ctf-tools/solve-scripts/solve_flag5.py 3) Or manually: base64 -d classified_doc.txt | python3 -c "import sys,codecs; print(codecs.decode(sys.stdin.read(),'rot13'))"

```
PS C:\Users\abhio\Downloads\NightFall\NightFall\ctf-tools\solve-scripts> python .\solve_flag5.py
╔═══════════════════════════════════════╗
  NightFall CTF — Flag 5: CIPHER NINE Solver
╚═══════════════════════════════════════╝

[*] Raw payload (from classified_doc.txt):
    WjFuaHRfRjRsbHtyMHQxM19wbHVzX2I2NF9jMXBoM3JfbjluM19ibDNwcjFudH0=

[+] After Base64 decode:
    Z1nht_F4ll{r0t13_plus_b64_c1ph3r_n9n3_bl3pr1nt}

[★] FLAG 5:
    M1aug_S4yy{e0g13_cyhf_o64_p1cu3e_a9a3_oy3ce1ag}

╔═══════════════════════════════════════╗
  M1aug_S4yy{e0g13_cyhf_o64_p1cu3e_a9a3_oy3ce1ag}
╚═══════════════════════════════════════╝
PS C:\Users\abhio\Downloads\NightFall\NightFall\ctf-tools\solve-scripts>
```

Flag 6 — Dead Signal

Flag: N1ght_F4ll{fr1d4_1nj3ct10n_k3y_extr4ct3d_l1v3}
Category: Frida Runtime Hook | Difficulty: ★★★★
Finding Method: Use Frida to hook runtime functions.
Concept: Dynamic instrumentation.
Explanation: The flag is assembled at runtime inside CryptoEngine.getMissionKey(). It is never stored statically, so it can only be extracted by hooking the method live with Frida and printing its return value.
Steps: 1) Start frida-server on a rooted emulator 2) Run: frida -U -f com.imf.nightfall -l ctf-tools/frida-scripts/flag6_dead_signal.js --no-pause 3) Navigate to Mission 06 inside the app 4) The Frida script intercepts getMissionKey() and prints the flag to the console



Flag 7 — Rogue Station

Flag: N1ght_F4ll{4es_ecb_m0l3_1d3nt1ty_CONTROL}
Category: AES SharedPreferences | Difficulty: ★★★★
Finding Method: Extract SharedPreferences and decrypt using AES.
Concept: Insecure local storage / AES-ECB cryptography.
Explanation: The flag is AES-ECB encrypted and stored in SharedPreferences (nightfall_session.xml). The AES key is hardcoded in SessionManager.kt, discoverable via JADX. The solve script combines both to decrypt the stored value.
Steps: 1) On a rooted emulator run: adb shell su -c "cat /data/data/com.imf.nightfall/shared_prefs/nightfall_session.xml" 2) Decompile with JADX and find the AES key in SessionManager.kt 3) Run: python3 ctf-tools/solve-scripts/solve_flag7.py with the extracted key and ciphertext

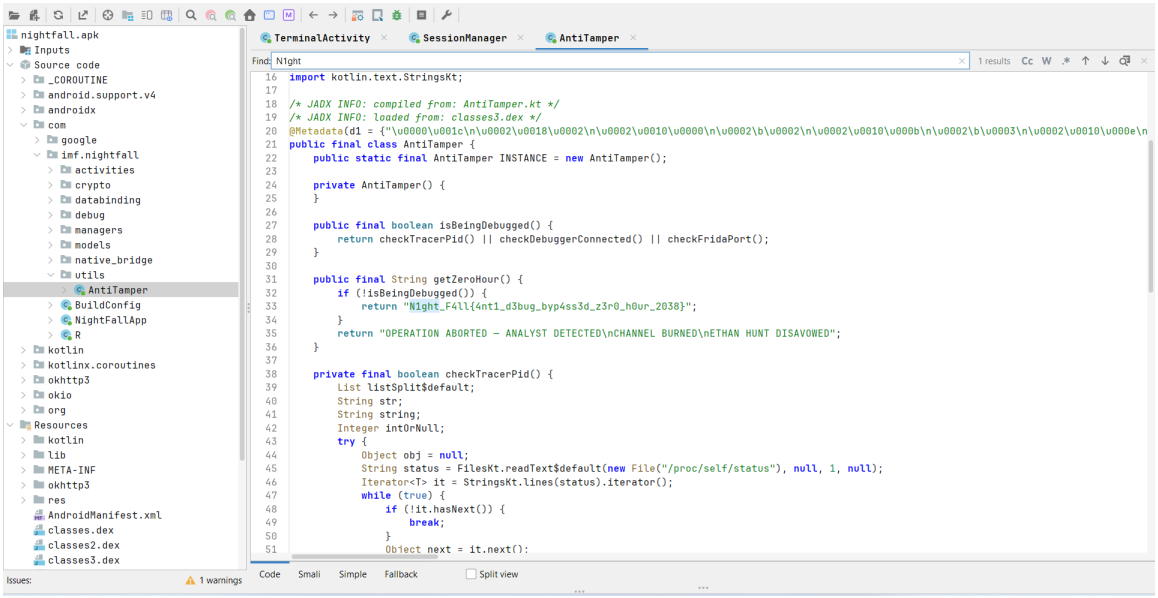
solve_flag5.py solve_flag7.py ×

NightFall > ctf-tools > solve-scripts > solve_flag7.py

```python
KNOWN_CIPHERTEXT_B64 = _encrypt_for_reference()


def _encrypt_for_reference() -> str:
    """
    Helper to pre-compute the expected ciphertext for verification.
    This runs AES/ECB encryption on the flag using the known key.
    """
    try:
        from Crypto.Cipher import AES
        from Crypto.Util.Padding import pad
        key    = b"IMF_K3y_S3cret16"
        flag   = b"N1ght_F4ll{4es_ecb_m0l3_1d3nt1ty_CONTROL}"
        cipher = AES.new(key, AES.MODE_ECB)
        ct     = cipher.encrypt(pad(flag, 16))
        return base64.b64encode(ct).decode()
    except Exception:
        return "ERROR_COMPUTING_REFERENCE"


def decrypt_flag(ciphertext_b64: str) -> str:
    try:
        ciphertext = base64.b64decode(ciphertext_b64)
    except Exception as e:
        return f"[-] Base64 decode failed: {e}"

    try:
        cipher    = AES.new(AES_KEY, AES.MODE_ECB)
        plaintext = unpad(cipher.decrypt(ciphertext), 16)
        return plaintext.decode("utf-8")
    except Exception as e:
        return f"[-] Decryption failed: {e}"
```

Flag 8 — Zero Hour

Flag: N1ght_F4ll{4nt1_d3bug_byp4ss3d_z3r0_h0ur_2038}
Category: Anti-Debug Bypass | Difficulty: ★★★★
Finding Method: Bypass anti-debugging checks using Frida.
Concept: Anti-debugging / anti-tamper bypass.
Explanation: AntiTamper.kt implements multiple debugger detection checks. If a debugger is detected the flag is withheld. The Frida script hooks all detection methods in AntiTamper and returns false, bypassing all checks and unlocking the flag.
Steps: 1) Start frida-server on a rooted emulator 2) Run: frida -U -f com.imf.nightfall -l ctf-tools/frida-scripts/flag8_zero_hour.js --no-pause 3) Navigate to Mission 08 in the app 4) The Frida script overrides all anti-debug guards and the flag is displayed



6. Conclusion

This project demonstrates key concepts in Android security including reverse engineering, encryption, runtime analysis, and network interception. It provides hands-on experience in identifying vulnerabilities and extracting sensitive data from mobile applications.